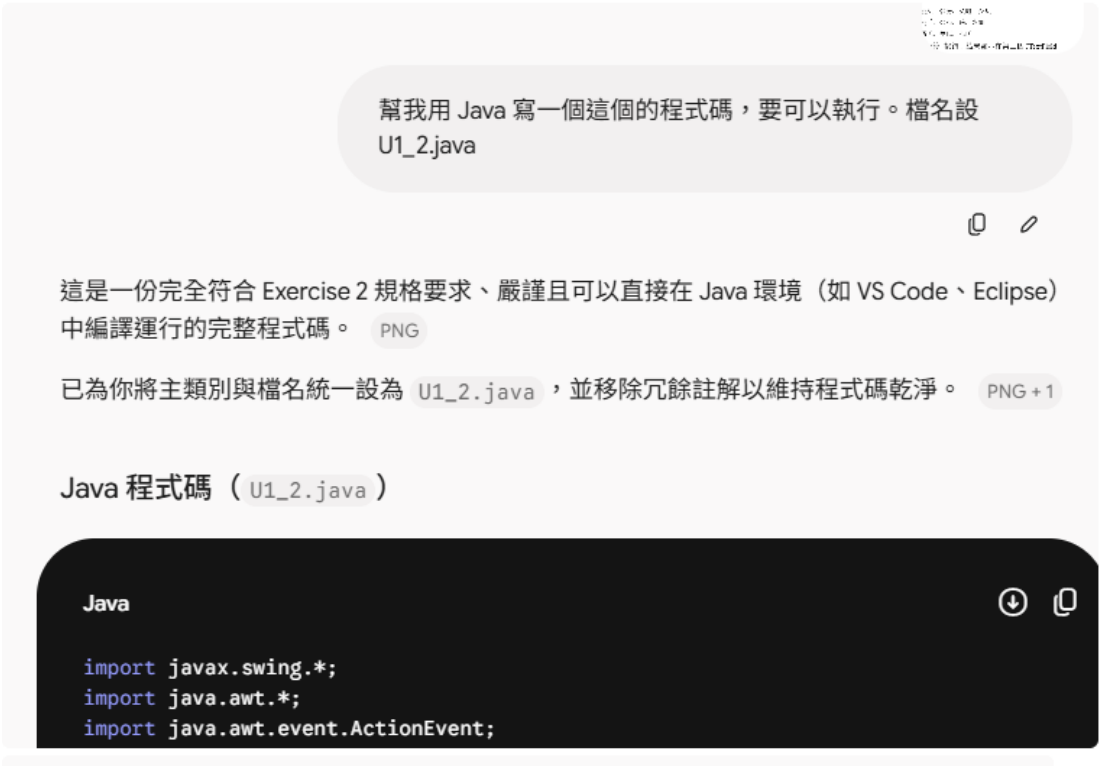
物件導向程式設計 2

A1143305 吳宥穎

U1_2 單位換算器

AI 對話紀錄



幫我用 Java 寫一個這個的程式碼，要可以執行。檔名設 U1_2.java

這是一份完全符合 Exercise 2 規格要求、嚴謹且可以直接在 Java 環境（如 VS Code、Eclipse）中編譯運行的完整程式碼。 PNG

已為你將主類別與檔名統一設為 `U1_2.java`，並移除冗餘註解以維持程式碼乾淨。 PNG +1

Java 程式碼（`U1_2.java`）

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
```



規格對照與結構解析：

1. **視窗佈局 (BorderLayout)**
   PNG
   - NORTH：置放 `JComboBox` 用於選擇換算類型（長度 / 重量 / 溫度）。 PNG
   - CENTER：採用 `GridLayout(2, 2)` 的兩列配置。第一行為「輸入框 (`JTextField`) + 來源單位下拉選單 (`JComboBox`)」；第二行為「結果顯示框 (`JTextField` ，不可編輯) + 目標單位下拉選單 (`JComboBox`)」。 PNG
   - SOUTH：置放「換算」按鈕 (`JButton`)。 PNG
2. **動態選單切換**
   PNG
   - 當切換頂部換算類型時，會觸發 `updateUnitOptions()` ，讓來源與目標下拉選單同步更新為該類別專屬的單位（長度：公尺、公分、英寸、英尺；重量：公斤、公克、磅、盎司；溫度：攝氏、華氏、克氏）。 PNG
3. **換算邏輯與安全防護**
   PNG

3. **換算邏輯與安全防護**
   PNG
   - **長度/重量**：先統一換算為基準單位（公尺 / 公克），再轉出為目標單位，計算準確且延伸性高。
   - **溫度**：支援攝氏（°C）、華氏（°F）與克氏（K）互相轉換。
   - **防呆處理**：使用 `try-catch` 捕捉使用者輸入非數字字串的情形，避免程式崩潰。
     PNG
4. **執行提示**
   PNG
   - 請將檔名設為 `U1_2.java`（與 `public class U1_2` 完全一致）。在 VS Code 中點擊右上角 **Run Java** 或按 `Ctrl + F5` 即可啟動。 PNG+1

Reflection

一、 哪裡有問題？

1. 若未合理劃分區域，輸入框與下拉選單容易排列混亂，無法呈現「輸入，選擇單位，輸出結果」的直覺視覺流程。

2. 若將所有單位（長度、重量、溫度）一次性全部列在下拉選單，使用者可能會選到「公尺轉公斤」這種不符合邏輯的無效組合。

3. 若沒有將結果欄設為唯讀，使用者可以直接修改結果內容；且若未對輸入字串進行防呆，輸入非數字字串（如英文字母）會導致程式崩潰。

二、 改了什麼？

1. 重構版面結構

- NORTH：放置頂層換算類型選單 。
- CENTER：採用 `GridLayout(2, 2)`，第一列放置「輸入框 + 來源單位選單」，第二列放置「輸出結果框 + 目標單位選單」。
- SOUTH：放置觸發換算動作的按鈕（`JButton`）。

2. 建立動態更新機制

- 加入事件監聽器，當頂部換算類型切換時，呼叫 `removeAllItems()` 清空舊選單，並自動載入該類別專屬的單位（如長度載入：公尺/公分/英寸/英尺）。

3. 強化防呆與換算邏輯

- 將輸出框設定為 `outputField.setEditable(false)`。

- 長度與重量轉換採用「輸入值 ， 基準單位（公尺/公克)， 目標單位」的兩階段轉換架構。

- 加入 `try-catch` 捕捉非數字輸入，並在輸出框提示「輸入錯誤」。

三、 為什麼要這樣改？

1. 提升使用者體驗與畫面對齊：利用 `GridLayout(2, 2)` 能確保輸入與輸出的文字框與選單完美對齊，符合傳統計算器與換算工具的操作習慣。

2. 確保邏輯正確性與資料一致性：透過選單連動，可以完全避免「跨類別換算」的無效操作發生，自動限制使用者只能在同類別的單位之間進行轉換。

3. 維護程式碼可擴充性與系統穩定度：引入「基準單位轉換法」能大幅減少換算邏輯中的條件判斷式數量；而加上 `setEditable(false)` 與 `try-catch` 則確保了 GUI 介面的嚴謹性與執行時的安全性。